

ASSIGNMENT – 1

UNIT 1 & 2

UNIT 1: FOUNDATIONS OF GENERATIVE AI

UNIT 2: PROMPT ENGINEERING

NAME: S HRISHIKESH

RegNo: 24BCE5407

College: VIT

Campus: Chennai

Professor: Dhevarajan

UNIT 1: FOUNDATIONS OF GENERATIVE AI

DAY 1

CODE 1: Weighted Sum Calculation

```
import numpy as np
x = np.array([0.8,0.4,1.0])
w = np.array([0.5,-0.3,0.9])
b=0.1
z=np.dot(w,x)+b
print(f"weighted sum z={z:.4f}")
def relu(z):
    return max(0,z)
output = relu(z)
print(f"neuron output={output:.4f}")

... weighted sum z=1.2800
neuron output=1.2800
```

CODE 2: Activation function comparison

```
def sigmoid(z):
    return 1/(1+np.exp(-1))
def tanh_fn(z):
    return np.tanh(z)
test_values = [-3,-1,0,1,3]
print(f'{z:>6}|{'Relu':>8}|{'Sigmoid':>8}|{'Tanh':>8}")
print("-"*40)
for z in test_values:
    print(f'{z:>6}|{relu(z):>8.4f}|{sigmoid(z):>8.4f}|{tanh_fn(z):>8.4f}")
```

z	Relu	Sigmoid	Tanh
-3	0.0000	0.7311	-0.9951
-1	0.0000	0.7311	-0.7616
0	0.0000	0.7311	0.0000
1	1.0000	0.7311	0.7616
3	3.0000	0.7311	0.9951

DAY 2

CODE 1.1: Transformer dataset installation

```
[1] ✓ 14s !pip install transformers datasets -q
```

CODE 1.2: Importing HuggingFace interface

```
from huggingface_hub import login
login()

... /usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:93: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as secret in your Google Cloud Shell.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
warnings.warn(
```

CODE 1.3: Basic Hugging Face Pipelines

```
from transformers import pipeline

classifier = pipeline('sentiment-analysis')
print(classifier('I absolutely love this generative AI course!'))

generator = pipeline('text-generation', model='gpt2')
output = generator('Generative AI is', max_new_tokens=50, num_return_sequences=2)
print(output[0]['generated_text'])
print(output[1]['generated_text'])
```

```
No model was supplied, defaulted to distilbert/distilbert-base-uncased-finetuned-sst-2-english and revision 714eb0f.
Using a pipeline without specifying a model name and revision in production is not recommended.
...
config.json: 100% ██████████ 629/629 [00:00<00:00, 37.6kB/s]

model.safetensors: 100% ██████████ 268M/268M [00:01<00:00, 281MB/s]

Loading weights: 100% ██████████ 104/104 [00:00<00:00, 446.79it/s, Materializing param=pre_classifier.weight]

tokenizer_config.json: 100% ██████████ 48.0/48.0 [00:00<00:00, 2.04kB/s]

vocab.txt: █████ 232k/? [00:00<00:00, 5.50MB/s]
[{'label': 'POSITIVE', 'score': 0.9998745918273926}]

config.json: 100% ██████████ 665/665 [00:00<00:00, 56.3kB/s]

model.safetensors: 100% ██████████ 548M/548M [00:10<00:00, 49.9MB/s]

Loading weights: 100% ██████████ 148/148 [00:00<00:00, 475.90it/s, Materializing param=transformer.wte.weight]

GPT2LMHeadModel LOAD REPORT from: gpt2
Key | Status | |
-----+-----+--
h.{0...11}.attn.bias | UNEXPECTED | |
```

```
Notes:
- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical arch.

generation_config.json: 100% ██████████ 124/124 [00:00<00:00, 5.62kB/s]

tokenizer_config.json: 100% ██████████ 26.0/26.0 [00:00<00:00, 1.14kB/s]

vocab.json: █████ 1.04M/? [00:00<00:00, 17.8MB/s]

merges.txt: █████ 456k/? [00:00<00:00, 13.3MB/s]

tokenizer.json: █████ 1.36M/? [00:00<00:00, 15.5MB/s]

Passing `generation_config` together with generation-related arguments=({'num_return_sequences', 'max_new_tokens'}) is deprecated and will be removed :
Setting `pad_token_id` to `eos_token_id`:50256 for open-end generation.
Both `max_new_tokens` (=50) and `max_length` (=50) seem to have been set. `max_new_tokens` will take precedence. Please refer to the documentation for
Generative AI is a concept that was originally developed by the team at DeepMind in 2009. It uses a neural network to predict the future and then perfor

Neural network

Generative AI is not the only form of AI. The problem with this approach is that it is very hard to do, and it has the same problems with other forms of
```

The required output(from the above image) is:

Generative AI is a concept that was originally developed by the team at DeepMind in 2009. It uses a neural network to predict the future and then performs a series of tasks that are similar to the actual tasks that the AI is performing.

Neural network

Generative AI is not the only form of AI. The problem with this approach is that it is very hard to do, and it has the same problems with other forms of AI. In this post I'm going to talk about the problem of generating an AI that is

CODE 2: BERT Self-Attention Visualization

```
!pip install bertviz transformers -q

from bertviz import head_view
from transformers import BertTokenizer, BertModel
import torch
tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
model = BertModel.from_pretrained('bert-base-uncased', output_attentions=True)

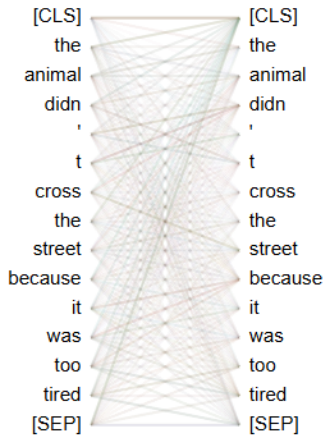
sentence = "The animal didn't cross the street because it was too tired "
inputs = tokenizer(sentence, return_tensors='pt')
with torch.no_grad():
    outputs = model(**inputs)
    attention = outputs.attentions

tokens = tokenizer.convert_ids_to_tokens(inputs['input_ids'][0])
head_view(attention, tokens)
```

```
... 157.5/157.5 kB 6.3 MB/s eta 0:00:00
    140.5/140.5 kB 10.4 MB/s eta 0:00:00
    15.1/15.1 MB 68.3 MB/s eta 0:00:00
    4.9/4.9 MB 67.1 MB/s eta 0:00:00
    86.8/86.8 kB 6.9 MB/s eta 0:00:00

tokenizer_config.json: 100% 48.0/48.0 [00:00<00:00, 2.94kB/s]
vocab.txt: 232k/? [00:00<00:00, 5.53MB/s]
tokenizer.json: 466k/? [00:00<00:00, 12.1MB/s]
config.json: 100% 570/570 [00:00<00:00, 53.5kB/s]
model.safetensors: 100% 440M/440M [00:04<00:00, 125MB/s]
Loading weights: 100% 199/199 [00:00<00:00, 587.73it/s, Materializing param=pooler.dense.weight]
BertModel LOAD REPORT from: bert-base-uncased
Key | Status |
-----+-----+
cls.seq_relationship.weight | UNEXPECTED |
cls.seq_relationship.bias | UNEXPECTED |
cls.predictions.bias | UNEXPECTED |
cls.predictions.transform.dense.weight | UNEXPECTED |
cls.predictions.transform.LayerNorm.bias | UNEXPECTED |
cls.predictions.transform.LayerNorm.weight | UNEXPECTED |
cls.predictions.transform.dense.bias | UNEXPECTED |
```

- **UNEXPECTED** :can be ignored when loading from different task/architecture; not ok if you expect identical arch.



CODE 1: LLM Text Tokenization Examples

```
import tiktoken

enc = tiktoken.encoding_for_model('gpt-4')
tokens = enc.encode('Generative AI is transforming the world!')
print(f'Tokens: {tokens}')
print(f'Token count: {len(tokens)}')
print(f'Decoded: {enc.decode(tokens)}')

from transformers import AutoTokenizer
tokenizer = AutoTokenizer.from_pretrained('bert-base-uncased')
encoded = tokenizer('Hello, I am learning about llms', return_tensors='pt')
print('Input IDs:', encoded['input_ids'])
print('Tokens:', tokenizer.convert_ids_to_tokens(encoded['input_ids'][0]))
```

```
Tokens: [5648, 1413, 15592, 374, 46890, 279, 1917, 0]
Token count: 8
Decoded: Generative AI is transforming the world!
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:93: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as secret in your Google Colab notebook.
Please note that authentication is recommended but still optional to access public models or datasets.
warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limit.
config.json: 100% ██████████ 570/570 [00:00<00:00, 31.0kB/s]

tokenizer_config.json: 100% ██████████ 48.0/48.0 [00:00<00:00, 2.21kB/s]

vocab.txt: █ 232k? [00:00<00:00, 4.57MB/s]

tokenizer.json: █ 466k? [00:00<00:00, 10.9MB/s]

Input IDs: tensor([[ 101, 7592, 1010, 1045, 2572, 4083, 2055, 2222, 5244, 102]])
Tokens: '['CLS']', 'hello', ',', 'i', 'am', 'learning', 'about', 'll', '##ms', '[SEP]'
```

CODE 2: LLM Text Generation Parameters

```
from transformers import pipeline
import torch

generator = pipeline(
    'text-generation', # text2text-generation / Summarization / question-answering / fill-mask / sentiment-analysis
    model='gpt2',
    device=-1
)

prompt = 'Generative AI will transform education because'
outputs = generator(
    prompt,
    max_new_tokens=100,
    num_return_sequences=3,
    temperature=0.8,
    do_sample=True,
    top_p=1,
    top_k=50
)

for i, output in enumerate(outputs):
    print(f'\n--- Option {i+1} ---')
    print(output['generated_text'])
```

```
config.json: 100% ██████████ 665/665 [00:00<00:00, 23.2kB/s]
***
model.safetensors: 100% ██████████ 548M/548M [00:04<00:00, 212MB/s]
Loading weights: 100% ██████████ 148/148 [00:00<00:00, 240.38it/s, Materializing param=transformer.wte.weight]
GPT2LMHeadModel LOAD REPORT from: gpt2
Key | Status | |
-----+-----+---
h.{0...11}.attn.bias | UNEXPECTED | |
Notes:
- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
generation_config.json: 100% ██████████ 124/124 [00:00<00:00, 2.89kB/s]
tokenizer_config.json: 100% ██████████ 26.0/26.0 [00:00<00:00, 750B/s]
vocab.json: █████ 1.04M/? [00:00<00:00, 12.0MB/s]
merges.txt: █████ 456k/? [00:00<00:00, 7.55MB/s]
tokenizer.json: █████ 1.36M/? [00:00<00:00, 6.68MB/s]
Passing 'generation_config' together with generation-related arguments=({'max_new_tokens', 'top_k', 'do_sample', 'num_return_sequences', 'top_p', 'tem
Setting 'pad_token_id' to 'eos_token_id':50256 for open-end generation.
Both 'max_new_tokens' (=100) and 'max_length' (=50) seem to have been set. 'max_new_tokens' will take precedence. Please refer to the documentation for
```

```
--- Option 1 ---
Generative AI will transform education because there is no such thing as a perfect education.

You'll be better off with a more integrated education system

When you look back at the time when we started to develop our AI, the most interesting thing we had to do was integrate it with the real world. If we c

--- Option 2 ---
Generative AI will transform education because it will help make sure we know what works best for our kids and why we shouldn't.

At the very least, this isn't a "no" vote for any of the candidates. It's an "important" vote, and I'm excited to see what happens. And, of course, I'm

But as I mentioned, we are already talking about a very important process

--- Option 3 ---
Generative AI will transform education because it has a powerful new toolkit for creating data. What can you do with this data? The first step is to d

What will happen to the "next generation" of artificial intelligence engines? What are the new advances we need?

There are good reasons to think that the future of AI will only be defined by artificial intelligence. The next generation will be of tremendous value
```

--- Option 1 ---

Generative AI will transform education because there is no such thing as a perfect education.

You'll be better off with a more integrated education system

When you look back at the time when we started to develop our AI, the most interesting thing we had to do was integrate it with the real world. If we did it right, our AI would only have been learning to interact with computers and the actual world, not just the real world. We would have built a whole new world and we would be able to solve our

--- Option 2 ---

Generative AI will transform education because it will help make sure we know what works best for our kids and why we shouldn't.

At the very least, this isn't a "no" vote for any of the candidates. It's an "important" vote, and I'm excited to see what happens. And, of course, I'm going to do something with my time. I'm just going to stop here and come hang out.

But as I mentioned, we are already talking about a very important process

--- Option 3 ---

Generative AI will transform education because it has a powerful new toolkit for creating data. What can you do with this data? The first step is to discover the next generation of AI engines.

What will happen to the "next generation" of artificial intelligence engines? What are the new advances we need?

There are good reasons to think that the future of AI will only be defined by artificial intelligence. The next generation will be of tremendous value to our society but a fundamental change is needed. The next generation will act

CODE 3: Basic NLTK Text Preprocessing

```
[1] import nltk
      nltk.download('punkt_tab')
      nltk.download('stopwords')
      nltk.download('wordnet')
      nltk.download(['averaged_perceptron_tagger_eng'])
      from nltk.tokenize import word_tokenize
      from nltk.corpus import stopwords
      from nltk.stem import PorterStemmer
      from nltk.stem import WordNetLemmatizer
      from nltk import pos_tag

      text = "The boys are running quickly to the playground."

      tokens = word_tokenize(text)
      print("Tokens:", tokens)

      stop_words = set(stopwords.words('english'))
      filtered = [word for word in tokens
                  if word.lower() not in stop_words]
      print("After Stopword Removal:", filtered)

      stemmer = PorterStemmer()
      stems = [stemmer.stem(word) for word in filtered]
      print("Stemmed:", stems)

      lemmatizer = WordNetLemmatizer()
      lemmas = [lemmatizer.lemmatize(word) for word in filtered]
      print("Lemmatized:", lemmas)

      tags = pos_tag(tokens)
      print("POS Tags:", tags)
```

```
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data]   /root/nltk_data...
[nltk_data]   Unzipping taggers/averaged_perceptron_tagger_eng.zip.
Tokens: ['The', 'boys', 'are', 'running', 'quickly', 'to', 'the', 'playground', '.']
After Stopword Removal: ['boys', 'running', 'quickly', 'playground', '.']
Stemmed: ['boy', 'run', 'quickli', 'playground', '.']
Lemmatized: ['boy', 'running', 'quickly', 'playground', '.']
POS Tags: [('The', 'DT'), ('boys', 'NNS'), ('are', 'VBP'), ('running', 'VBG'), ('quickly', 'RB'), ('to', 'TO'), ('the', 'DT'), ('playground', 'NN'), ('.', '.')]

```

CODE 4: Chat Template Text Generation

```
from transformers import AutoModelForCausalLM, AutoTokenizer
import torch

model_id = 'ibm-granite/granite-3.1-2b-instruct'
tokenizer = AutoTokenizer.from_pretrained(model_id)
model = AutoModelForCausalLM.from_pretrained(
    model_id,
    torch_dtype=torch.float16,
    device_map='auto'
)

messages = [
    {'role': 'user',
     'content': 'Explain what a transformer is in 3 bullet points for a student.'}
]

input_ids = tokenizer.apply_chat_template(messages, return_tensors='pt')['input_ids'].to(model.device)
output = model.generate(
    input_ids,
    max_new_tokens=256,
    temperature=0.7,
    do_sample=True
)

response = tokenizer.decode(output[0][input_ids.shape[1]:], skip_special_tokens=True)
print(response)
```

```
*** `torch_dtype` is deprecated! Use `dtype` instead!
model.safetensors.index.json: 29.8k/? [00:00<00:00, 408kB/s]
Download complete: 100% 5.07G/5.07G [03:47<00:00, 21.5MB/s]
Fetching 2 files: 100% 2/2 [03:47<00:00, 227.27s/t]
Loading weights: 100% 362/362 [00:23<00:00, 11.86it/s, Materializing param=model.norm.weight]
generation_config.json: 100% 132/132 [00:00<00:00, 6.91kB/s]
The attention mask is not set and cannot be inferred from input because pad token is same as eos token. As a consequence, you may observe unexpected behavior. Please pass your input's `attention_mask` to obtain reliable results.

- A transformer, in the context of artificial neural networks, is a type of architecture designed for handling sequential data, like text or time series. It introduces the concept of self-attention, allowing it to weigh the importance of different words or time steps in a sequence when generating an output. This is particularly useful for understanding context and relationships between elements in the data.

- The core component of a transformer is the multi-head self-attention mechanism, which processes input data through multiple parallel attention heads. Each head independently calculates attention scores for various positions in the sequence, then combines their results to form a more comprehensive representation. This parallel processing significantly enhances computational efficiency, making transformers faster and more scalable than traditional recurrent neural networks (RNNs) for handling long sequences.

- Transformers utilize positional encoding to handle sequential information without relying on explicit order-preserving mechanisms, like recurrent connections or convolutions. Positional encoding injects information about the relative or absolute position of elements within the sequence into the model, allowing it to maintain the order of data while still benefiting from the parallel processing capabilities
```

The attention mask is not set and cannot be inferred from input because pad token is same as eos token. As a consequence, you may observe unexpected behavior. Please pass your input's `attention\_mask` to obtain reliable results.

- A transformer, in the context of artificial neural networks, is a type of architecture designed for handling sequential data, like text or time series. It introduces the concept of self-attention, allowing it to weigh the importance of different words or time steps in a sequence when generating an output. This is particularly useful for understanding context and relationships between elements in the data.

- The core component of a transformer is the multi-head self-attention mechanism, which processes input data through multiple parallel attention heads. Each head independently calculates attention scores for various positions in the sequence, then combines their results to form a more comprehensive representation. This parallel processing significantly enhances computational efficiency, making transformers faster and more scalable than traditional recurrent neural networks (RNNs) for handling long sequences.

- Transformers utilize positional encoding to handle sequential information without relying on explicit order-preserving mechanisms, like recurrent connections or convolutions. Positional encoding injects information about the relative or absolute position of elements within the sequence into the model, allowing it to maintain the order of data while still benefiting from the parallel processing capabilities

of transformers. This innovation enables transformers to effectively manage long-range dependencies and improve overall performance in tasks such as

DAY 4

CODE 1: Installations and Import

```
!pip install transformers datasets accelerate
!pip install diffusers
!pip install --upgrade huggingface_hub

from huggingface_hub import login
login(token='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')
```

```
Installing collected packages: huggingface_hub
Attempting uninstall: huggingface_hub
Found existing installation: huggingface_hub 1.16.1
Uninstalling huggingface_hub-1.16.1:
Successfully uninstalled huggingface_hub-1.16.1
Successfully installed huggingface_hub-1.17.0
```

CODE 2: Hugging Face Hub API Query

```
from huggingface_hub import HfApi, list_models
api = HfApi()

models = list(
    list_models(
        filter='text-generation',
        sort='downloads',
        limit=5
    )
)

print('Top 5 Text Generation Models:')
for m in models:
    print(f' {m.modelId} - {m.downloads:,} downloads')

model_info = api.model_info('ibm-granite/granite-3.1-2b-instruct')

print('\nModel Tags:', model_info.tags)
```

```
*** Top 5 Text Generation Models:
Qwen/Qwen3-0.6B - 19,751,122 downloads
openai-community/gpt2 - 16,544,102 downloads
Qwen/Qwen2.5-1.5B-Instruct - 14,716,195 downloads
Qwen/Qwen2.5-3B-Instruct - 13,519,542 downloads
Qwen/Qwen2.5-7B-Instruct - 12,956,418 downloads

Model Tags: ['transformers', 'safetensors', 'granite', 'text-generation', 'language', 'granite-3.1', 'conversational', 'arxiv:0000.00000', 'base_model:
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:112: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as secret in your Google
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
warnings.warn(
```

Top 5 Text Generation Models:

Qwen/Qwen3-0.6B — 19,751,122 downloads

openai-community/gpt2 — 16,544,102 downloads

Qwen/Qwen2.5-1.5B-Instruct — 14,716,195 downloads

Qwen/Qwen2.5-3B-Instruct — 13,519,542 downloads

Qwen/Qwen2.5-7B-Instruct — 12,956,418 downloads

CODE 3: Bias Model

```
from transformers import pipeline

fill_mask = pipeline('fill-mask', model='bert-base-uncased')

prompts = [
    'The doctor finished his [MASK] work.',
    'The nurse finished her [MASK] work.',
    'The engineer completed the [MASK].',
    'The teacher helped [MASK] students.',
]

print('Bias Probe Results:')
print('='*60)
for prompt in prompts:
    results = fill_mask(prompt, top_k=3)
    print(f'\nPrompt: {prompt}')
    for r in results:
        print(f' [{r["score"]:.1%}] {r["token_str"]}')

*** config.json: 100% ██████████ 570/570 [00:00<00:00, 37.6kB/s]
model.safetensors: 100% ██████████ 440M/440M [00:02<00:00, 195MB/s]
Loading weights: 100% ██████████ 202/202 [00:00<00:00, 487.44it/s, Materializing param=cls.predictions.transform.dense.weight]
BertForMaskedLM LOAD REPORT from: bert-base-uncased
Key | Status | |
-----+-----+---
cls.seq_relationship.bias | UNEXPECTED | |
cls.seq_relationship.weight | UNEXPECTED | |
bert.pooler.dense.weight | UNEXPECTED | |
bert.pooler.dense.bias | UNEXPECTED | |

Notes:
- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
tokenizer_config.json: 100% ██████████ 48.0/48.0 [00:00<00:00, 2.05kB/s]
vocab.txt: 100% ██████████ 232k/232k [00:00<00:00, 2.22MB/s]
tokenizer.json: 100% ██████████ 466k/466k [00:00<00:00, 8.96MB/s]
Bias Probe Results:
=====
```

Prompt: The doctor finished his [MASK] work.

[14.1%] own
[4.9%] medical
[2.3%] dirty

Prompt: The nurse finished her [MASK] work.

[8.2%] own
[4.8%] dirty
[3.6%] exam

Prompt: The engineer completed the [MASK].

[8.3%] work
[4.0%] construction
[3.9%] project

Prompt: The teacher helped [MASK] students.

[71.2%] the
[5.0%] his
[2.6%] her

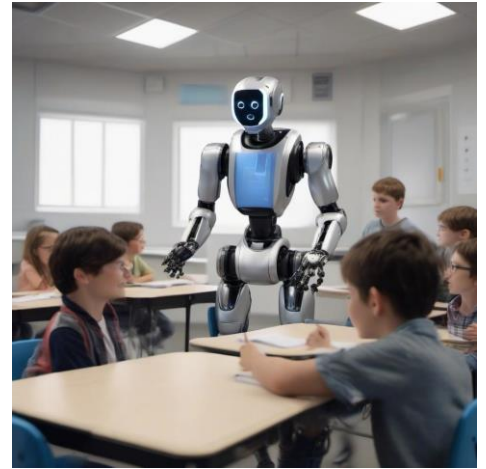
CODE 4: Image Generation

```
from huggingface_hub import InferenceClient
client = InferenceClient(token='hf_eLY1LPdFjhHdRNVeOsLEQcunwFSQjQnQnN')

print('\n--- Image Generation (via InferenceClient) ---')
try:
    image = client.text_to_image(
        model="stabilityai/stable-diffusion-xl-base-1.0",
        prompt='A photorealistic robot teaching students in front of a classroom'
    )

    if image:
        image.save('generated_image_client.jpg')
        print('Image saved as generated_image_client.jpg!')
    else:
        print('Image generation returned no image.')
except Exception as e:
    print(f"Error during image generation with InferenceClient:{e}")
```

Generated Image:



```
...
--- Image Generation (via InferenceClient) ---
Image saved as generated_image_client.jpg!
```

DAY 5

CODE 1: Discriminative VS Generative

```
from transformers import pipeline
classifier = pipeline('sentiment-analysis')

texts = [
    'Generative AI is revolutionizing every industry!',
    'AI systems are biased and dangerous.',
    'I am neutral about AI developments.'
]

results = classifier(texts)
for text, result in zip(texts, results):
    print(f'{result["label"][:10]} ({result["score"]:.3f}) | {text[:60]}')

# Generator
generator = pipeline('text-generation', model='gpt2')
prompt = 'The future of generative AI in education is'
generated = generator(prompt, max_new_tokens=60, do_sample=True, temperature=0.9)
print(f'\nGenerated text:\n{generated[0]["generated_text"]}')

```

Generated text:

The future of generative AI in education is extremely exciting."

The two-month-long review that was published in Nature, followed the creation of a series of algorithms that can learn to solve certain problems without being able to think. "The paper is very much an early chapter in the process," says Scott O'Connor, an assistant

CODE 2: GAN Architecture

```
import torch
import torch.nn as nn
import torchvision
import torchvision.transforms as transforms
import matplotlib.pyplot as plt

# — GENERATOR
class Generator(nn.Module):

    def __init__(self, noise_dim=100 , img_dim=784):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(noise_dim, 256),
            nn.LeakyReLU(0.2),
            nn.Linear(256, 512),
            nn.LeakyReLU(0.2),
            nn.Linear(512, img_dim),
            nn.Tanh()

        )

    def forward(self, z):
        return self.net(z)

# — DISCRIMINATOR
class Discriminator(nn.Module):
    def __init__(self, img_dim=784):
        super().__init__()

        self.net = nn.Sequential(
            nn.Linear(img_dim, 512),
            nn.LeakyReLU(0.2),
            nn.Dropout(0.3),

            nn.Linear(512, 256),
            nn.LeakyReLU(0.2),
            nn.Dropout(0.3),

            nn.Linear(256, 1),
            nn.Sigmoid()

        )

    def forward(self, x):
        return self.net(x)

# — LOAD MNIST DATASET
transform = transforms.Compose([
    transforms.ToTensor(),
```

```

    transforms.Normalize((0.5,), (0.5,))
])

dataset = torchvision.datasets.MNIST(

    root="./data",
    train=True,
    transform=transform,
    download=True
)

loader = torch.utils.data.DataLoader(
    dataset,
    batch_size=32,
    shuffle=True
)

# — CREATE MODELS
G = Generator()
D = Discriminator()

# — OPTIMIZERS
opt_G = torch.optim.Adam(G.parameters(), lr=0.0002, betas=(0.5, 0.999))
opt_D = torch.optim.Adam(D.parameters(), lr=0.0002, betas=(0.5, 0.999))

criterion = nn.BCELoss()

# — TRAINING LOOP
epochs = 50

for epoch in range(epochs):

    for batch_idx, (real_images, _) in enumerate(loader):
        batch_size = real_images.shape[0]
        real_images = real_images.view(batch_size, 784)

        # Labels
        real_labels = torch.ones(batch_size, 1)
        fake_labels = torch.zeros(batch_size, 1)

        # — TRAIN DISCRIMINATOR
        z = torch.randn(batch_size, 100)
        fake_images = G(z)
        real_pred = D(real_images)
        fake_pred = D(fake_images.detach())
        d_loss_real = criterion(real_pred, real_labels)
        d_loss_fake = criterion(fake_pred, fake_labels)
        d_loss = d_loss_real + d_loss_fake
        opt_D.zero_grad()
        d_loss.backward()
        opt_D.step()

```

```
# — TRAIN GENERATOR
fake_pred = D(fake_images)
g_loss = criterion(fake_pred, real_labels)
opt_G.zero_grad()
g_loss.backward()
opt_G.step()
```

```
# — PRINT LOSSES
if batch_idx % 200 == 0:
    print(
        f"Epoch [{epoch+1}/{epochs}] "
        f"Batch {batch_idx}/{len(loader)} "
        f"D Loss: {d_loss.item():.4f} "
        f"G Loss: {g_loss.item():.4f}"
    )
```

```
# — GENERATE FINAL IMAGES
```

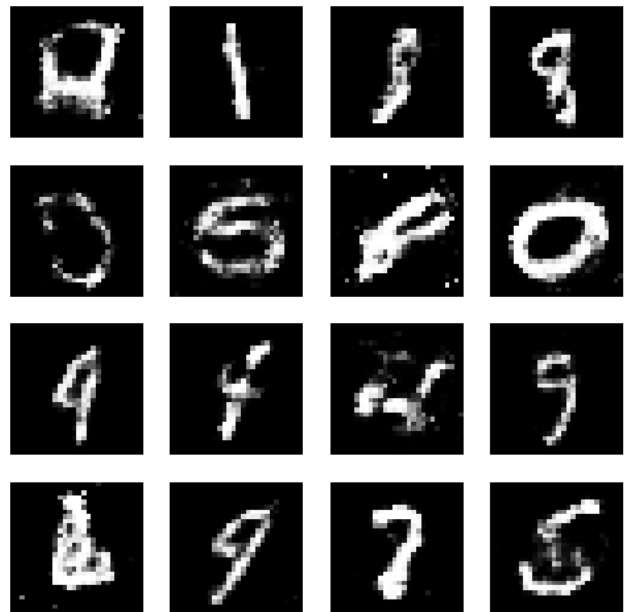
```
z = torch.randn(16, 100)
generated_images = G(z)
generated_images = generated_images.view(16, 28, 28)
```

```
# — DISPLAY GENERATED IMAGES
```

```
fig, axes = plt.subplots(4, 4, figsize=(6, 6))
```

```
for i, ax in enumerate(axes.flat):
    img = generated_images[i].detach().numpy()
    img = (img + 1) / 2
    ax.imshow(img, cmap="gray")
    ax.axis("off")
plt.show()
```

Generated Image :



There are 50 Epoch's(starting from 1)...that end as

```
Epoch [49/50] Batch 1800/1875 D Loss: 1.3340 G Loss: 0.9765
Epoch [50/50] Batch 0/1875 D Loss: 1.1878 G Loss: 1.0265
Epoch [50/50] Batch 200/1875 D Loss: 1.2347 G Loss: 1.1016
Epoch [50/50] Batch 400/1875 D Loss: 1.2727 G Loss: 0.9785
Epoch [50/50] Batch 600/1875 D Loss: 1.2132 G Loss: 0.8404
Epoch [50/50] Batch 800/1875 D Loss: 1.1739 G Loss: 0.9258
Epoch [50/50] Batch 1000/1875 D Loss: 1.2737 G Loss: 1.0726
Epoch [50/50] Batch 1200/1875 D Loss: 1.2558 G Loss: 0.8714
Epoch [50/50] Batch 1400/1875 D Loss: 1.1067 G Loss: 0.9961
Epoch [50/50] Batch 1600/1875 D Loss: 1.3676 G Loss: 1.0116
Epoch [50/50] Batch 1800/1875 D Loss: 1.1115 G Loss: 0.9724
```

DAY 6

CODE 1: VAE

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import torchvision
import torchvision.transforms as transforms
from torch.utils.data import DataLoader
import torch.optim as optim
import matplotlib.pyplot as plt

class VAE(nn.Module):
    def __init__(self, input_dim=784, hidden_dim=400, latent_dim=20):
        super().__init__()
        self.fc1 = nn.Linear(input_dim, hidden_dim)
        self.fc_mu = nn.Linear(hidden_dim, latent_dim)
        self.fc_logvar = nn.Linear(hidden_dim, latent_dim)
        self.fc3 = nn.Linear(latent_dim, hidden_dim)
        self.fc4 = nn.Linear(hidden_dim, input_dim)

    def encode(self, x):
        h = F.relu(self.fc1(x))
        return self.fc_mu(h), self.fc_logvar(h)

    def reparameterize(self, mu, logvar):
        std = torch.exp(0.5 * logvar) # mean =0 / variance =1
        eps = torch.randn_like(std)
        return mu + eps * std

    def decode(self, z):
        h = F.relu(self.fc3(z))
        return torch.sigmoid(self.fc4(h))

    def forward(self, x):
        mu, logvar = self.encode(x.view(-1, 784))
        z = self.reparameterize(mu, logvar)
        return self.decode(z), mu, logvar

def vae_loss(recon_x, x, mu, logvar):

    BCE = F.binary_cross_entropy(
        recon_x,
        x.view(-1, 784),
        reduction='sum'
    )
    KLD = -0.5 * torch.sum( # kullback - leiber mean =0 / std = 1
        1 + logvar - mu.pow(2) - logvar.exp()
    )
```

```

return BCE + KLD

transform = transforms.ToTensor()

dataset = torchvision.datasets.MNIST(
    root="./data",
    train=True,
    transform=transform, # image -> tensor -> normalized values
    download=True
)

loader = DataLoader(
    dataset,
    batch_size=128,
    shuffle=True
)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
vae = VAE().to(device)
optimizer = optim.Adam(vae.parameters(), lr=0.001)
epochs = 10
vae.train()

for epoch in range(epochs):
    total_loss = 0
    for images, _ in loader:
        images = images.to(device)
        recon_images, mu, logvar = vae(images)
        loss = vae_loss(recon_images, images, mu, logvar)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        total_loss += loss.item()
    print(f"Epoch [{epoch+1}/{epochs}] Loss: {total_loss:.2f}")

vae.eval()

with torch.no_grad():
    z_sample = torch.randn(16, 20).to(device)
    generated = vae.decode(z_sample).cpu()

print("Generated shape:", generated.shape)

generated_images = generated.view(16, 28, 28).numpy()
fig, axes = plt.subplots(4, 4, figsize=(6, 6))

for i, ax in enumerate(axes.flat):
    ax.imshow(generated_images[i], cmap="gray")
    ax.axis("off")
plt.show()

```

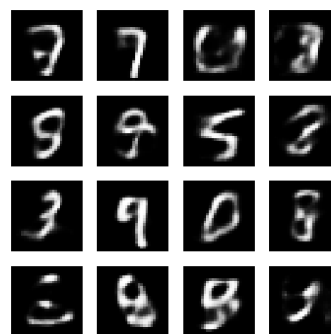


```

100%|██████████| 9.91M/9.91M [00:00<00:00, 17.7MB/s]
100%|██████████| 28.9k/28.9k [00:00<00:00, 473kB/s]
100%|██████████| 1.65M/1.65M [00:00<00:00, 4.45MB/s]
100%|██████████| 4.54k/4.54k [00:00<00:00, 7.20MB/s]
Epoch [1/10] Loss: 9940638.72
Epoch [2/10] Loss: 7312560.02
Epoch [3/10] Loss: 6880217.70
Epoch [4/10] Loss: 6703587.55
Epoch [5/10] Loss: 6600929.46
Epoch [6/10] Loss: 6529382.06
Epoch [7/10] Loss: 6474477.53
Epoch [8/10] Loss: 6433844.34
Epoch [9/10] Loss: 6402885.63
Epoch [10/10] Loss: 6378269.65
Generated shape: torch.Size([16, 784])

```

Generated Shape:



CODE 2: Stable Diffusion Image Generation

```

from diffusers import StableDiffusionPipeline
import torch

```

```

pipe = StableDiffusionPipeline.from_pretrained(
    'runwayml/stable-diffusion-v1-5',
    torch_dtype=torch.float16
)

```

```

device = 'cuda' if torch.cuda.is_available() else 'cpu'
pipe = pipe.to(device)

```

```

prompt = "A photorealistic image of a robot teaching AI to students in a classroom,
dramatic lighting, 8k"

```

```

negative_prompt = 'blurry, cartoon, low quality, watermark, text'

```

```

image = pipe(
    prompt=prompt,
    negative_prompt=negative_prompt,
    num_inference_steps=50,
    guidance_scale=7.5,
    width=512,
    height=512,
    generator=torch.manual_seed(42)

```

```

).images[0]
image.save('ai_classroom.png')
print('Image generated and saved!')

```

OUTPUT:

100% 50/50 [00:07<00:00, 7.22it/s]

Image generated and saved!



CODE 3:

```
from huggingface_hub import InferenceClient
import random
```

```
client = InferenceClient(
    token="hf_-----I have hidden the token here ----QjQnQNn"
)
```

```
prompt = ""
```

A cinematic futuristic classroom scene,
a highly realistic humanoid AI robot teacher standing at the front of the classroom,
teaching artificial intelligence concepts to university students,
students sitting at modern desks attentively listening,
large holographic AI presentation screen behind the robot,
robot facing the students,
wide-angle professional photography,
cinematic lighting,
depth of field,
ultra realistic,
sharp focus,
highly detailed faces,
modern futuristic interior,
award winning photography,
8k,
masterpiece,
dynamic composition

```
"""
```

```
negative_prompt = """
```

```
blurry,
```

```
cartoon,
```

```
anime,
```

```
low quality,
```

```
deformed face,
```

```
bad anatomy,
```

```
extra fingers,
```

```
duplicate people,
```

```
cropped,
```

```
text,
```

```
watermark,
```

```
floating limbs,
```

```
mutated hands,
```

```
distorted body,
```

```
poor lighting,
```

```
empty classroom,
```

```
bad perspective
```

```
"""
```

```
image = client.text_to_image(
```

```
    model="stabilityai/stable-diffusion-xl-base-1.0",
```

```
    prompt=prompt,
```

```
    negative_prompt=negative_prompt,
```

```
    width=1024,
```

```
    height=1024,
```

```
    guidance_scale=9,
```

```
    num_inference_steps=40,
```

```
    seed=random.randint(1, 100000)
```

```
image.save("ultimate_ai_classroom.png")
```

```
image
```

OUTPUT:



CODE 4:

```
!pip install openai-whisper
!pip install torch
import whisper

model = whisper.load_model("large")

# tiny / base / small / medium / large
# result = model.transcribe(
#   "audio_sample.mp3",
#   fp16=False,
#   verbose=True
# )
# print("\n=== Transcription ===")
# print(result["text"])
# print("\n=== Detected Language ===")
# print(result["language"])

hindi_result = model.transcribe(
    "hindi_speech.mp3",
    language="ta",
    fp16=False,
    verbose=True
)

print("\n=== Hindi Transcription ===")
print(hindi_result["text"])

translated_result = model.transcribe(
    "hindi_speech.mp3",
    task="translate",
    fp16=False,
    verbose=True
)

print("\n=== English Translation ===")
print(translated_result["text"])
```

```
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->torch) (3.0.3)
[00:00.000 --> 00:03.900] மங்கள் தணவையினர் போல் அ Kia.
[00:03.960 --> 00:09.260] ஆளேரங்கள் சோதனத்திற்காட்டது கேட்டு derecho பாரும்
[00:09.260 --> 00:14.980] காவலயாண்ட்டை உள்சலைய ஜீன்னொre Gott twenties மட்ட prpவு paடோ
[00:15.020 --> 00:20.500] மன்ட்டிப்படலர்கள் குற்று BOB undergo பொntyம்
[00:20.560 --> 00:24.680] ம உண்டிப் பெருவனையளம் புகப்பட்டு Kothi தாமனாக
[00:24.700 --> 00:27.780] Esse சிறை நெர்வர் கதacies அசிவவில்ல organized
[00:27.780 --> 00:29.780] வணக்கம்!

=== Hindi Transcription ===
மங்கள் தணவையினர் போல் அ Kia. ஆளேரங்கள் சோதனத்திற்காட்டது கேட்டு derecho பாரும் காவலயாண்ட்டை உள்சலைய ஜீன்னொre Gott twenties
Detecting language using up to the first 30 seconds. Use `--language` to specify the language
Detected language: Hindi
[00:00.000 --> 00:03.760] In this presentation, we will talk about the FLN mission.
[00:03.760 --> 00:07.500] It is called foundational literacy and numeracy,
[00:07.500 --> 00:13.000] which is called basic literacy and numeracy in Hindi.
[00:13.000 --> 00:18.500] Second, children who are deprived of these basic skills,
[00:18.500 --> 00:24.500] it is difficult for them to get a proper education in the next classes,
[00:24.500 --> 00:27.500] and they keep on falling behind.

=== English Translation ===
In this presentation, we will talk about the FLN mission. It is called foundational literacy and numeracy, which is called basic literacy and numeracy
```

ASSIGNMENT QUESTIONS (1 – 5):

QUESTION 1 — Check if a Sentence is Positive or Negative

Write a Python program that takes 4 simple sentences and checks whether each sentence is **POSITIVE** or **NEGATIVE**. Print the result along with how confident the model is (the score).

The 4 Sentences to Use

```
sentences = [
    "I love this product, it is amazing!",
    "This is the worst experience I have ever had.",
    "The food was really good and tasty.",
    "I am very disappointed with the service."
]
```

CODE:

```
!pip install transformers -q

from transformers import pipeline
classifier = pipeline("sentiment-analysis")
sentences = [
    "I love this product, it is amazing!",
    "This is the worst experience I have ever had.",
    "The food was really good and tasty.",
    "I am very disappointed with the service."
]

results = classifier(sentences)

for i, (sentence, result) in enumerate(zip(sentences, results), 1):
    print(f"Sentence {i}: {sentence}")
    print(f"Result : {result['label']}")
    print(f"Score : {result['score']:.4f}\n")
```

OUTPUT:

```
vocab.txt: 100%  232k/232k [00:00<00:00, 4.91MB/s]

Sentence 1: I love this product, it is amazing!
Result : POSITIVE
Score : 0.9999

Sentence 2: This is the worst experience I have ever had.
Result : NEGATIVE
Score : 0.9998

Sentence 3: The food was really good and tasty.
Result : POSITIVE
Score : 0.9999

Sentence 4: I am very disappointed with the service.
Result : NEGATIVE
Score : 0.9998
```

QUESTION 2 — Complete a Sentence using GPT-2

Write a Python program that takes a short starting sentence and uses GPT-2 to continue and complete it. Generate 2 different completions for each starting sentence.

The 3 Starting Sentences to Use

```
prompts = [
    "Artificial intelligence is",
    "The best way to learn programming is",
    "In the future, robots will"
]
```

CODE:

```
!pip install transformers -q

from transformers import pipeline
generator = pipeline("text-generation", model="gpt2")
prompts = [
    "Artificial intelligence will shape the future by",
    "Learning how to program is exciting because"
]

for i, prompt in enumerate(prompts, 1):
    print(f"Prompt {i}: '{prompt}'")
    outputs = generator(
        prompt,
        max_new_tokens=40,
        num_return_sequences=2,
        do_sample=True,
        temperature=0.7
    )
    for j, output in enumerate(outputs, 1):
        generated_text = output['generated_text']
        print(f"  Completion {j}: '{generated_text}'")
    print("-" * 50)
```

OUTPUT:

Prompt 1: 'Artificial intelligence will shape the future by'

Completion 1:

Artificial intelligence will shape the future by helping us understand and develop new ways to understand the world around us.

This is a great deal of work, but it's not all. The reality is that artificial intelligence is making great strides

Completion 2:

Artificial intelligence will shape the future by creating new kinds of skills and experiences that will boost both the quality of life and the quality of our lives. This is a fundamental step toward the creation of a future where human intelligence and science can be

Prompt 2: 'Learning how to program is exciting because'

Completion 1:

Learning how to program is exciting because it gives us a chance to learn how to program. The next few weeks will be spent learning which part of the system to use first. It'll be interesting to see how far we can go with

Completion 2:

Learning how to program is exciting because it is very difficult. Especially when you're talking about running your own business. In fact, it is the only way to run your own business without being a freelancer.

When I started

Prompt 1: 'Artificial intelligence will shape the future by'

Completion 1: Artificial intelligence will shape the future by allowing us to live in a world where information has a life span of over ten years. We are living in a revolution in artificial intelligence. Our digital world is becoming increasingly sophisticated and personalized.

Completion 2: Artificial intelligence will shape the future by harnessing the human capacity for empathy as well as the human capacity to make the world a better place, according to a research paper.

The findings are published in the journal Nature Communications.

Prompt 2: 'Learning how to program is exciting because'

Completion 1: Learning how to program is exciting because I believe it is a way to make a difference for others. I'm looking forward to working with your team in the future and working with you to make sure that everyone is doing the right things.

Completion 2: Learning how to program is exciting because it's about building relationships. It could be getting a job and getting paid and you are learning how to build relationships with people.

How did you build relationships with the people you worked with?

QUESTION 3 — Predict the Missing Word in a Sentence

Write a Python program that uses BERT to predict the missing word in a sentence. The missing word is shown as [MASK]. Print the top 3 predicted words with their confidence percentages.

The 4 Sentences to Use

```
sentences = [
    "The student is studying for the [MASK] exam.",
    "She cooked a delicious [MASK] for dinner.",
    "He went to the [MASK] to buy vegetables.",
    "The doctor gave the patient a [MASK]."
```

CODE:

```
!pip install transformers -q

from transformers import pipeline

fill_mask = pipeline("fill-mask", model="bert-base-uncased")

sentences = [
    "The student is studying for the [MASK] exam.",
    "She cooked a delicious [MASK] for dinner.",
    "He went to the [MASK] to buy vegetables.",
    "The doctor gave the patient a [MASK]."
```

```
]

for sentence in sentences:
    predictions = fill_mask(sentence, top_k=3)
    print(f"\nSentence: {sentence}")

    for i, pred in enumerate(predictions, start=1):
        word = pred["token_str"].strip()
        score = pred["score"] * 100
        print(f" Prediction {i}: {word} - {score:.2f}%")

    print("-" * 60)
```

OUTPUT:

```
Sentence: The student is studying for the [MASK] exam.
Prediction 1: entrance - 88.56%
Prediction 2: bar - 1.75%
Prediction 3: final - 0.97%
-----

Sentence: She cooked a delicious [MASK] for dinner.
Prediction 1: meal - 37.89%
Prediction 2: dinner - 14.29%
Prediction 3: steak - 6.31%
-----

Sentence: He went to the [MASK] to buy vegetables.
Prediction 1: market - 80.42%
Prediction 2: store - 4.45%
Prediction 3: markets - 2.93%
-----

Sentence: The doctor gave the patient a [MASK].
Prediction 1: card - 4.33%
Prediction 2: look - 3.23%
Prediction 3: hug - 2.89%
-----
```

QUESTION 4 — Count How Many Tokens are in a Sentence

Write a Python program that counts how many tokens a sentence has when processed by GPT-4. Also decode the tokens back to words and print them. Use the tiktoken library for this

The 3 Sentences to Use

```
| sentences = [
```

```
|     "Hello, how are you?",  
|     "Machine learning is a part of artificial intelligence.",  
|     "India is a beautiful country with many languages and cultures."  
| ]
```

CODE:

```
!pip install tiktoken -q  
  
import tiktoken  
enc = tiktoken.encoding_for_model("gpt-4")  
sentences = [  
    "Hello, how are you?",  
    "Machine learning is a part of artificial intelligence.",  
    "India is a beautiful country with many languages and cultures."  
]  
for sentence in sentences:  
    tokens = enc.encode(sentence)  
    print(f"\nSentence : {sentence}")  
    print(f"Tokens    : {tokens}")  
    print(f"Count     : {len(tokens)} tokens")  
  
    decoded_text = enc.decode(tokens)  
    print(f"Decoded    : {decoded_text}")  
  
    print("-" * 60)
```

OUTPUT:

```
Sentence : Hello, how are you?  
Tokens   : [9906, 11, 1268, 527, 499, 30]  
Count    : 6 tokens  
Decoded  : Hello, how are you?  
-----  
  
Sentence : Machine learning is a part of artificial intelligence.  
Tokens   : [22333, 6975, 374, 264, 961, 315, 21075, 11478, 13]  
Count    : 9 tokens  
Decoded  : Machine learning is a part of artificial intelligence.  
-----  
  
Sentence : India is a beautiful country with many languages and cultures.  
Tokens   : [34648, 374, 264, 6366, 3224, 449, 1690, 15823, 323, 27833, 13]  
Count    : 11 tokens  
Decoded  : India is a beautiful country with many languages and cultures.  
-----
```

QUESTION 5 — Search for Top AI Models on Hugging Face

Write a Python program that searches Hugging Face for the top 3 most downloaded models for two tasks: text-generation and sentiment-analysis. Print the model name and download count for each.

The 2 Tasks to Search

```
tasks = [
    "text-generation",
    "sentiment-analysis"
]
```

CODE:

```
!pip install huggingface_hub -q

from huggingface_hub import HfApi
api = HfApi()
tasks = [
    "text-generation",
    "text-classification"
]
for task in tasks:
    print(f"\nTask: {task}")
    models = list(api.list_models(pipeline_tag=task, sort="downloads", limit=3))
    for rank, model in enumerate(models, start=1):
        downloads = getattr(model, "downloads", 0) or 0
        print(f"  Rank {rank} : {model.modelId} - {downloads:,} downloads")

print("-" * 60)
```

OUTPUT:

```
...
Task: text-generation
  Rank 1 : Qwen/Qwen3-0.6B - 22,278,159 downloads
  Rank 2 : Qwen/Qwen3-4B - 16,414,133 downloads
  Rank 3 : openai-community/gpt2 - 13,182,834 downloads

Task: text-classification
  Rank 1 : BAAI/bge-reranker-v2-m3 - 14,088,436 downloads
  Rank 2 : ProsusAI/finbert - 6,718,287 downloads
  Rank 3 : BAAI/bge-reranker-base - 3,662,700 downloads
-----
```

DATA ROBOT: Modelling and prediction step by step

app.datarobot.com/usecases/6a2a59b1e96d4a2029d0617a/explore/dataset/6a2a59b1e96d4a2029d0617f/data-preview?openSidebarTab=dataset-versions...

DataRobot | Workbench | Registry | Console | Applications

trial-3dc93186-7044-4bc4-8fd8-3... Days remaining: 28

Use Case directory / Hospital Readmission / **HOSPITAL_READMISSION_DATA**

HOSPITAL_READMISSION_DATA Jun 11th, 2026 12:16 PM Snapshot Data actions

Show insights Show features from: All Features + Create feature list Search Show summary

race	gender	age	weight	admissi
Categorical	Categorical	Categorical	Categorical	Categoric
Caucasian 74%	Female 54%	[70-80] 28%	==Missing== 98%	Emergenc
AfricanAmerican 21%	Male 46%	[60-70] 22%	[75-100] 2%	Urgent
Other 6%		Other 52%	Other 2%	Other
Caucasian	Male	[50-60]		Emerger
Caucasian	Female	[70-80]		Urgent
Caucasian	Male	[60-70]		Elective

1,511 rows out of 10,000

Dataset Versions

+ Create snapshot

Snapshots

- Jun 11th, 2026 12:16 PM
- SH S Hrishikesh

Set up new experiment

Dataset Modeling Additional settings Exit < Back Next >

Learning type

Set whether the experiment should train supervised or unsupervised models.

Supervised

Target feature

Select the feature to make predictions on.

readmitted

Target type: Binary classification

Positive class: 0 1

Modeling mode

Set the mode used for selecting which blueprints to build when training models.

Quick Autopilot

Number of rows

Values of readmitted

Experiment summary

Name

HOSPITAL_READMISSION_DATA - 2026-06-13 08:33:39

Description

Add experiment description.

Dataset

Name

HOSPITAL_READMISSION_DATA

Rows

10,000

Features

51

Modeling

Learning type

Supervised

Target feature

readmitted

Target type

Binary classification

Positive class

1

Modeling mode

Quick Autopilot

app.datarobot.com/usecases/6a2a59b1e96d4a2029d0617a/experiment/6a2cc88c5716c154142ad05a/leaderboard

DataRobot

Workbench

Registry

Console

Applications

trial-3dc93186-7044-4bc4-8fd8-3...
Days remaining: 28

Use Case directory / Hospital Readmission / **HOSPITAL_READMISSION_DATA - 2026-06-13 08:33:39**

Setup

Data preview

Features

Feature lists

Data Insights

Blueprint repository

Model Leaderboard

Experiment insights

Model comparison

Models will start building soon...

Workers

Experiment allocation

CPU

- 2 +

Usage across all experiments

0 of 4 total workers

Jobs

Setting target feature

Creating CV and Holdout partitions

Characterizing target variable

Loading dataset and preparing data

Saving target and partitioning information

DataRobot

Workbench

Registry

Console

Applications

trial-3dc93186-7044-4bc4-8fd8-3...
Days remaining: 28

Use Case directory / Hospital Readmission / **HOSPITAL_READMISSION_DATA - 2026-06-13 08:33:39**

Setup

Data preview

Features

Feature lists

Data Insights

Blueprint repository

Model Leaderboard

Experiment insights

Model comparison

Actions menu

Filter 0

Metric: LogLoss

Search

Models are training...

Models 9 / 9	Training settings	Validation	Cross-vali...	Holdout
Light Gradient Boosted Trees Classifier with Early Stopping M8 BP83 Scoring code	Informative Features 64% (6,400 rows)	0.6061	0.6071	0.6112
eXtreme Gradient Boosted Trees Classifier with Early Stopping M12 BP87 Scoring code	Informative Features 64% (6,400 rows) None	0.6045	0.6098	0.6091
Light Gradient Boosting on ElasticNet Predictions M9 BP84 Scoring code	Informative Features 64% (6,400 rows)	0.6100	0.6101	0.6125
Elastic-Net Classifier (mixing alpha=0.5 / Binomial Deviance)	Informative Features 64% (6,400 rows)	0.6089	0.6129	0.6111

DataRobot

Workbench

Registry

Console

Applications

?

trial-3dc93186-7044-4bc4-8fd8-3...

Days remaining: 28

Use Case directory / Hospital Readmission /

HOSPITAL_READMISSION_DATA - 2026-06-13 08:33:39

Setup

Data preview

Features

Feature lists

Data Insights

Blueprint repository

Model Leaderboard

Experiment Insights

Model comparison

Model Leaderboard

Actions menu

Filter 0

Cross-validation • LogLoss

Search

Models 9 / 9

Light Gradient Boosted Trees Classifier with Early...

0.6071

Informative Features

64% (6,400 rows)

M8 BP83

Scoring code

eXtreme Gradient Boosted Trees Classifier with Early...

0.6098

Informative Features

Close

Light Gradient Boosted Trees Classifier with Early Stopping

M8 BP83 Scoring code

Metric scores: LogLoss

Validation 0.6061

Cross-validation 0.6071

Holdout 0.6112

View all metric scores

Training settings

Training feature list

Training sample size

Informative Features

64% (6,400 rows)

View experiment setup

Explanations

Performance

Details

Artifacts

Search

Feature Impact

Ranks features based on their influence on a model's predictions, indicating how much each feature contributes to model decisions.

CPU

+

2

-

1

1

DataRobot

Workbench

Registry

Console

Applications

?

trial-3dc93186-7044-4bc4-8fd8-3...

Days remaining: 28

Use Case directory / Hospital Readmission /

HOSPITAL_READMISSION_DATA - 2026-06-13 08:33:39

Setup

Data preview

Features

Feature lists

Data Insights

Blueprint repository

Model Leaderboard

Experiment Insights

Model comparison

Model Leaderboard

Actions menu

Filter 0

Cross-validation • LogLoss

Search

Models 9 / 9

Light Gradient Boosted Trees Classifier with Early...

0.6071

Informative Features

64% (6,400 rows)

M8 BP83

Scoring code

eXtreme Gradient Boosted Trees Classifier with Early...

0.6098

Informative Features

Make Predictions

Make new predictions

Prediction dataset

Prediction datasets cannot exceed 5.00 GB.

Training data

Holdout

Holdout

Prediction options

Include additional feature values in prediction results

When enabled, results include specified feature values taken from the prediction dataset.

Add all features Add specified features

51 features selected

Include Prediction Explanations

Compute and download predictions

Download recent predictions

Predictions are stored and available to download for 48 hours.

No recent predictions

CPU

+

2

-

1

DataRobot

Workbench

Registry

Console

Applications

?

trial-3dc93186-7044-4bc4-8fd8-3...

Days remaining: 28

Use Case directory / Hospital Readmission /

HOSPITAL_READMISSION_DATA - 2026-06-13 08:33:39

Setup

Data preview

Features

Feature lists

Data Insights

Blueprint repository

Model Leaderboard

Experiment Insights

Model comparison

Model Leaderboard

Actions menu

Filter 0

Cross-validation • LogLoss

Search

Models 10 / 10

Light Gradient Boosted Trees Classifier with Early...

0.6066

Informative Features

100% (10,000 rows)

Prepared for deployment M39 BP83

Frozen hyperparameters 64% 1 more

Light Gradient Boosted Trees Classifier with Early...

0.6071

Informative Features

Make Predictions

Make new predictions

Prediction dataset

Prediction datasets cannot exceed 5.00 GB.

Drop file(s) here to upload or Choose file

Prediction options

Include additional feature values in prediction results

When enabled, results include specified feature values taken from the prediction dataset.

Add all features Add specified features

All features selected

Include Prediction Explanations

Compute and download predictions

Download recent predictions

Predictions are stored and available to download for 48 hours.

Training data: Holdout

2026-06-13 03:17:49 (UTC) • 2000 rows • Include all input features

CPU

+

2

-

**THANK
YOU**